

Documentación de Arquitectura — MeritCoin

Formato ARC42 — Versión 0.3.0

Proyecto: MeritCoin — Sistema de Recompensas Académicas Digitales
Institución: Universidad Tecnológica de Bolívar
Fecha: Abril 2026
Estado: En desarrollo activo (Fase 7 de 10 completada)
Rama principal de desarrollo: `feature/transacciones-base`

1. Introducción y Objetivos

1.1 Descripción del sistema

MeritCoin es un sistema de incentivos académicos que integra la plataforma LMS Moodle con tecnología blockchain. Permite que los profesores configuren reglas de recompensa por actividad o curso, y que los estudiantes acumulen tokens digitales (MRT) e insignias verificables on-chain al completar logros académicos.

El sistema opera de forma híbrida: la lógica de negocio y configuración vive off-chain (Moodle + PostgreSQL), mientras que la emisión de tokens e insignias queda registrada permanentemente on-chain (Ethereum EVM compatible).

1.2 Objetivos de calidad

Prioridad	Atributo	Descripción
1	Integridad	Cada evento académico debe producir exactamente un registro on-chain. Duplicados son rechazados por <code>event_id</code> único.
2	Trazabilidad	Todo evento queda registrado en la cola Moodle, en el <code>audit_log</code> PostgreSQL y en la blockchain. Tres capas de auditoría independientes.
3	Seguridad	Toda comunicación Moodle → Backend está firmada con HMAC-SHA256. Los contratos usan <code>AccessControl</code> con roles explícitos.
4	Extensibilidad	El sistema debe poder desplegarse en SAVIO (instancia Moodle de la universidad) con cambios únicamente en la capa de presentación.
5	Operabilidad	El backend debe poder conectarse a cualquier nodo EVM compatible cambiando únicamente la variable <code>BLOCKCHAIN_RPC_URL</code> .

1.3 Partes interesadas (Stakeholders)

Rol	Interés principal
Estudiante	Acumular y consultar monedas e insignias obtenidas en sus cursos

Rol	Interés principal
Profesor	Configurar reglas de recompensa por curso/actividad sin conocimiento técnico de blockchain
Administrador Moodle	Instalar y configurar el plugin; gestionar credenciales del backend
Desarrollador	Mantener y extender el sistema; desplegar en SAVIO

2. Restricciones de Arquitectura

2.1 Restricciones técnicas

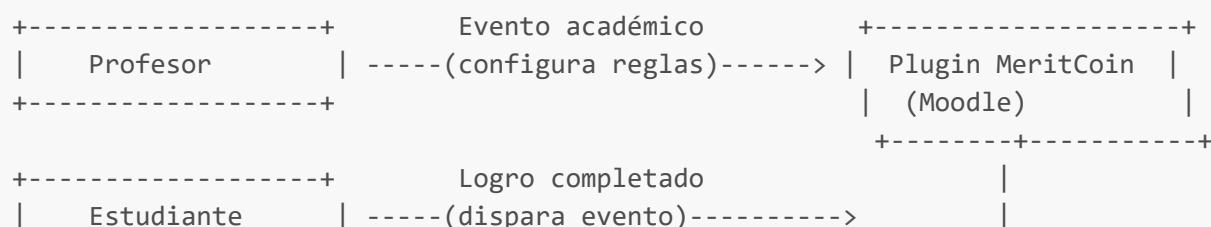
Restricción	Razón
El plugin debe seguir la Moodle Plugin API estándar	Compatibilidad con Moodle 4.x y con SAVIO
Los contratos usan únicamente OpenZeppelin 5.x (sin librerías de pago)	Licencia MIT, auditadas, sin dependencias propietarias
El backend corre dentro de Docker ; el nodo blockchain corre en la máquina host	Restricción de entorno de desarrollo en Windows con Docker Desktop
No se almacenan datos personales on-chain	Privacidad: solo wallets e IDs ofuscados viajan a la blockchain
El .env de la raíz es la única fuente de configuración	config.py del backend apunta a ../../.env

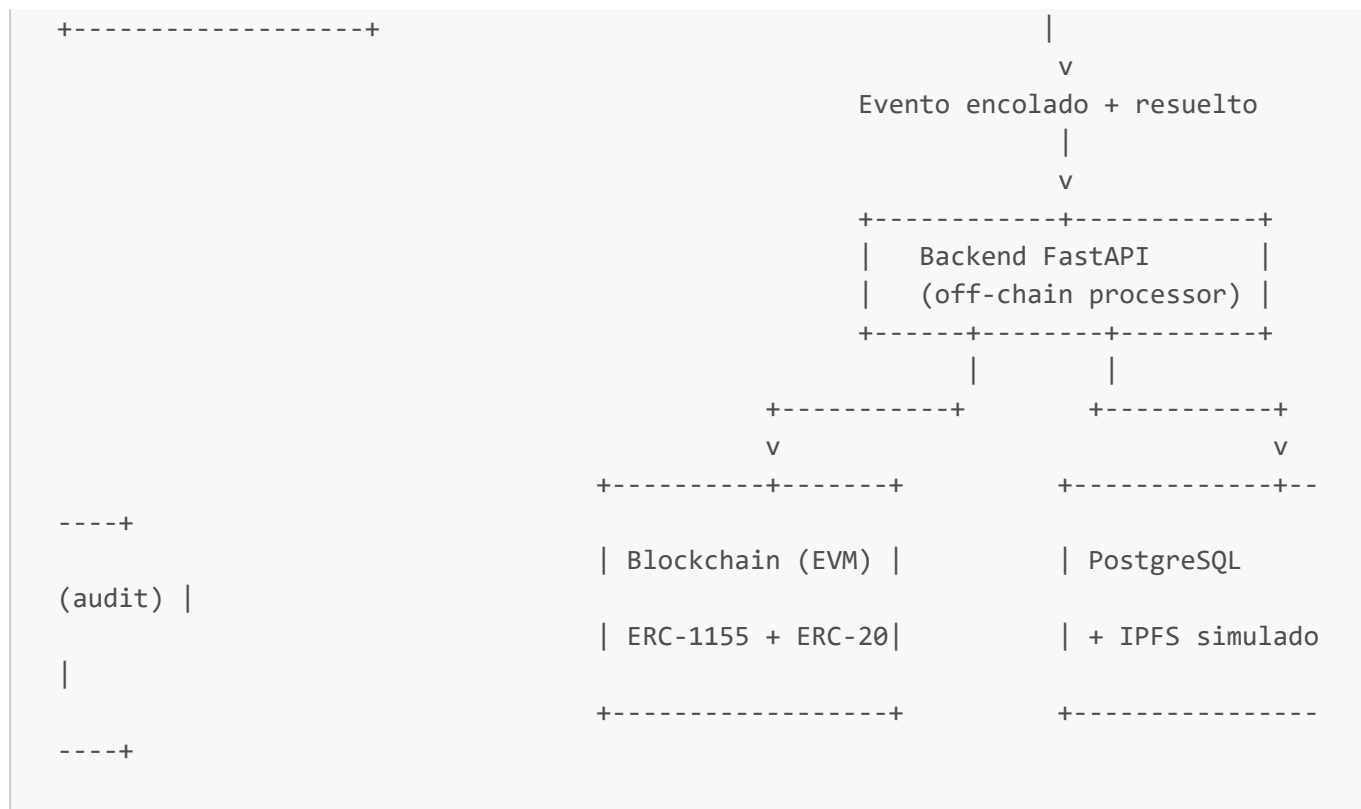
2.2 Restricciones organizacionales

- El proyecto es académico; la infraestructura de producción target es **SAVIO** (Moodle institucional de la UTB).
- El ajuste visual para SAVIO debe poder hacerse sin reescribir lógica de negocio.
- Las claves privadas usadas en desarrollo (**0xac0974...**) son las cuentas públicas de Hardhat; nunca deben usarse en producción.

3. Contexto del Sistema

3.1 Contexto de negocio





3.2 Contexto técnico

Canal	Protocolo	Descripción
Moodle → Backend	HTTP POST + HMAC-SHA256	Envío de eventos académicos firmados
Backend → Blockchain	JSON-RPC (web3.py)	Llamadas a <code>mintBadge</code> y <code>mint</code> en los contratos
Backend → PostgreSQL	asyncpg (SQLAlchemy async)	Persistencia del <code>audit_log</code>
Plugin → MariaDB	Moodle DBAL	Persistencia de queue, rules, earnings, spend
Estudiante → Moodle	HTTPS (navegador)	Acceso al dashboard de insignias y monedas

4. Estrategia de Solución

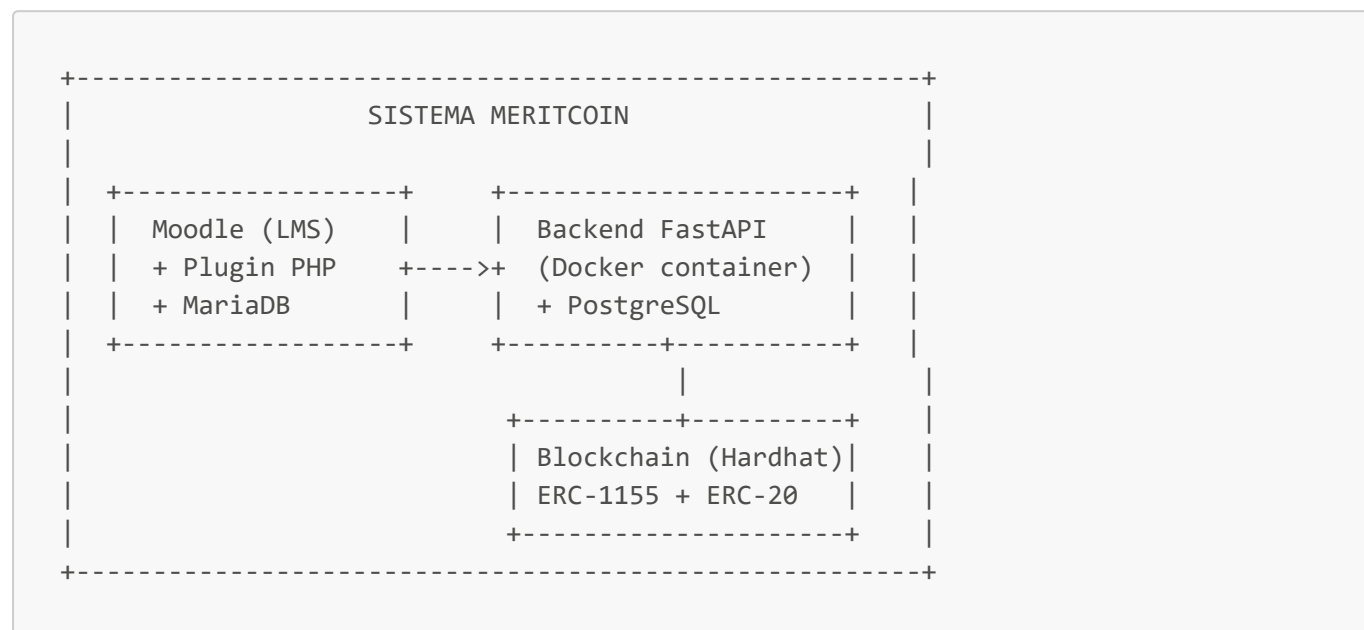
La solución separa en tres capas de responsabilidad claramente delimitadas:

- Capa de configuración y captura (Moodle + Plugin PHP):** El profesor define las reglas de recompensa. El observer captura los eventos del LMS y calcula el valor de monedas antes de encolar. Esta capa no tiene dependencias directas de la blockchain.
- Capa de procesamiento off-chain (FastAPI + PostgreSQL):** Recibe eventos firmados, verifica integridad, genera metadatos OBv2, simula pin IPFS y orquesta las llamadas a la blockchain. Garantiza idempotencia mediante `event_id` único.
- Capa de registro permanente (Contratos Solidity + EVM):** Emite badges ERC-1155 y tokens ERC-20. Es la fuente de verdad final e inmutable del sistema.

La decisión de calcular `coins_amount` en el plugin (no en el backend) permite que el backend sea agnóstico a las reglas de negocio del LMS, facilitando la futura integración con otros sistemas distintos de Moodle.

5. Vista de Bloques

5.1 Nivel 1 — Sistema completo



5.2 Nivel 2 — Plugin Moodle (caja blanca)

Componente	Responsabilidad
<code>observer.php</code>	Escucha eventos Moodle (<code>course_completed</code> , <code>grade_item_created</code>) y dispara el procesamiento
<code>rules_service.php</code>	Consulta <code>local_meritcoin_rules</code> y resuelve cuántas monedas corresponden al evento
<code>send_events_task.php</code>	Tarea programada (Moodle Task API) que envía eventos <code>pending</code> al backend vía HTTP
<code>api_client.php</code>	Encapsula la comunicación HTTP con el backend; genera firma HMAC-SHA256
<code>manage.php</code> + <code>editrule.php</code>	UI del profesor para CRUD de reglas por curso
<code>dashboard.php</code>	UI del estudiante: saldo MRT e insignias por curso
<code>rule_form.php</code>	Formulario Moodle (Form API) para creación/edición de reglas

5.3 Nivel 2 — Backend FastAPI (caja blanca)

5.4 Nivel 2 — Contratos Solidity (caja blanca)

Ambos contratos heredan `AccessControl` (roles `ISSUER_ROLE`, `MINTER_ROLE`) y `Pausable` de OpenZeppelin 5.x.

6. Vista de Ejecución

6.1 Escenario principal — Evento de completación de curso

5 / 10

6.2 Escenario de idempotencia — Evento duplicado

Si el backend recibe un `event_id` que ya existe en `audit_log`, retorna `200 OK` con el mensaje "Evento ya fue procesado anteriormente" sin volver a llamar a la blockchain. El plugin marca el evento como `processed` igualmente.

6.3 Escenario de wallet no registrada

Si el estudiante no tiene wallet configurada en su perfil Moodle, el observer encola el evento con estado `pending_wallet` en lugar de `pending`. La tarea programada ignora estos eventos hasta que el estudiante registre su wallet.

7. Vista de Despliegue

7.1 Entorno de desarrollo (actual)

6 / 10

```
└─ Procesos nativos
   └─ npx hardhat node --hostname 0.0.0.0 (puerto 8545)
```

Comunicación Docker → Host: los contenedores usan `host.docker.internal` para alcanzar el nodo Hardhat en la máquina host. La variable `BLOCKCHAIN_RPC_URL=http://host.docker.internal:8545` es obligatoria en este entorno.

7.2 Entorno objetivo — SAVIO (producción)

```
Servidor UTB
├─ SAVIO (Moodle institucional)
│   └─ Plugin local_meritcoin instalado
├─ Backend FastAPI
│   └─ Apuntando a nodo EVM de producción (testnet pública o red privada)
└─ Nodo EVM
    └─ Hardhat en modo fork de testnet, o red privada Besu/Geth
```

En SAVIO, `BLOCKCHAIN_RPC_URL` apuntará al nodo EVM institucional. El resto de la arquitectura no cambia; únicamente se ajustan variables de entorno y los templates visuales del plugin.

8. Conceptos Transversales

8.1 Seguridad

- **HMAC-SHA256:** cada petición del plugin al backend incluye una firma calculada con `HMAC_SECRET` compartido. El backend rechaza con `401` cualquier petición con firma inválida.
- **Roles en contratos:** `ISSUER_ROLE` controla `mintBadge`; `MINTER_ROLE` controla `mint`. Solo el deployer del backend tiene estos roles asignados.
- **Pausable:** ambos contratos pueden pausarse ante incidentes sin necesidad de redesplicue.
- **sesskey:** todas las acciones de escritura del plugin (crear/editar/borrar reglas) requieren `require_sesskey()` de Moodle para prevenir CSRF.

8.2 Idempotencia

El campo `event_id` en `audit_log` (PostgreSQL) tiene índice único. Si el backend intenta insertar un `event_id` duplicado, la BD lanza una excepción que el servicio captura y convierte en respuesta `200` sin reintentar la transacción blockchain.

8.3 Trazabilidad en tres capas

Capa	Almacén	Qué registra
Plugin (Moodle)	<code>local_meritcoin_queue</code>	Estado del evento: pending → processed/failed

Capa	Almacén	Qué registra
Backend (off-chain)	PostgreSQL <code>audit_log</code>	event_id, wallet, txHash badge, txHash MRT, CID IPFS, timestamp
Blockchain (on-chain)	EVM	Transacciones inmutables de <code>mintBadge</code> y <code>mint</code>

8.4 Metadatos Open Badges v2 (OBv2)

Cada insignia emitida lleva metadatos conformes al estándar OBv2:

- `name`, `description`, `image` del curso/actividad
- `recipient` (wallet del estudiante, ofuscado con hash SHA-256)
- `issuedOn` (timestamp del evento)
- `verification` (tipo blockchain + dirección del contrato)

El CID IPFS actual es simulado (`QmSimulated...`). En producción se sustituirá por un pin real a nodo IPFS o Pinata.

8.5 Ledger de saldo por curso

El saldo MRT gastable de un estudiante en un curso se calcula en el plugin como:

```
saldo_disponible = SUM(local_meritcoin_earnings.coins_earned WHERE userid,
courseid)
- SUM(local_meritcoin_spend.coins_spent WHERE userid, courseid)
```

Esta lógica es independiente por curso, lo que permite que un estudiante tenga saldos y reglas diferentes en cada uno de sus cursos.

9. Decisiones de Arquitectura (ADR)

ADR-001: Cálculo de monedas en el plugin, no en el backend

Contexto: Las reglas de recompensa son configuradas por el profesor en Moodle. Se evaluó si el cálculo de `coins_amount` debía hacerse en el plugin o en el backend.

Decisión: El plugin resuelve las reglas y envía `coins_amount` ya calculado al backend.

Consecuencias:

- (+) El backend es agnóstico a las reglas de Moodle; puede integrarse con otros LMS.
- (+) Las reglas se pueden cambiar sin redeploy del backend.
- (-) El backend confía en el valor de monedas que envía el plugin; no lo valida de forma independiente.

ADR-002: Comunicación síncrona Moodle → Backend vía cola interna

Contexto: Los eventos de Moodle ocurren en tiempo real durante la sesión del usuario.

Decisión: El observer encola el evento en MariaDB inmediatamente y la tarea programada lo procesa de forma asíncrona cada X minutos.

Consecuencias:

- (+) El usuario no experimenta latencia de la blockchain durante su sesión.
- (+) Si el backend no está disponible, los eventos quedan en cola para reintentar.
- (-) Existe un retardo entre el logro académico y la emisión on-chain.

ADR-003: Doble base de datos (MariaDB + PostgreSQL)

Contexto: Moodle usa MariaDB de forma nativa; el backend necesita su propia BD.

Decisión: MariaDB exclusivamente para datos del plugin Moodle. PostgreSQL exclusivamente para el audit_log del backend.

Consecuencias:

- (+) Separación de responsabilidades; el backend puede funcionar sin acceso a MariaDB.
- (+) PostgreSQL ofrece mejor soporte para queries analíticas (futuro dashboard administrativo).
- (-) Dos motores de BD en el stack aumentan la complejidad operacional.

ADR-004: IPFS simulado en desarrollo

Contexto: Integrar un nodo IPFS real añade complejidad al entorno de desarrollo.

Decisión: El `badges_service` genera un CID simulado (`QmSimulated...`) en lugar de hacer un pin real.

Consecuencias:

- (+) El entorno de desarrollo es más simple y reproducible.
- (-) Los metadatos OBv2 no son accesibles públicamente en desarrollo; habrá que reemplazar antes del despliegue en SAVIO.

10. Riesgos y Deuda Técnica

ID	Tipo	Descripción	Impacto	Plan de mitigación
R-01	Riesgo	IPFS simulado en producción invalida la verificabilidad de las insignias OBv2	Alto	Integrar Pinata o nodo IPFS propio antes del despliegue en SAVIO (Fase 10)
R-02	Riesgo	Clave privada del deployer en <code>.env</code> ; si se filtra, un atacante puede mintear tokens arbitrariamente	Alto	Usar HSM o cuenta multisig con Gnosis Safe en producción
R-03	Deuda técnica	Los tests de backend (pytest) no cubren el nuevo flujo con	Medio	Revisar y actualizar en la siguiente iteración

ID	Tipo	Descripción	Impacto	Plan de mitigación
		<code>rules_service</code> ; pueden tener aserciones desactualizadas		
R-04	Riesgo	El nodo Hardhat es local; no es un entorno de testnet pública	Medio	Migrar a Sepolia o Polygon Mumbai antes de SAVIO
R-05	Deuda técnica	El dashboard del estudiante (<code>dashboard.php</code>) está en desarrollo; aún no consume el endpoint <code>/summary</code>	Bajo	Completar en Fase 8
R-06	Deuda técnica	No existe mecanismo de reintentos explícito para eventos <code>failed</code> en la cola	Bajo	Implementar lógica de retry con backoff en <code>send_events_task.php</code>

Documento generado en Abril 2026 — MeritCoin v0.3.0 — Universidad Tecnológica de Bolívar